



Programmiervorkurs

Einführung in Java – Tag 1

Ablauf



Montag – Donnerstag

Zeitraum	Was?
10 – 12 Uhr	Informationen, Übungsbesprechung, Vorlesung
12 – 13 Uhr	Mittagspause
ab 13 Uhr	Übungen, gemütliches Zusammensitzen

Inhaltsübersicht



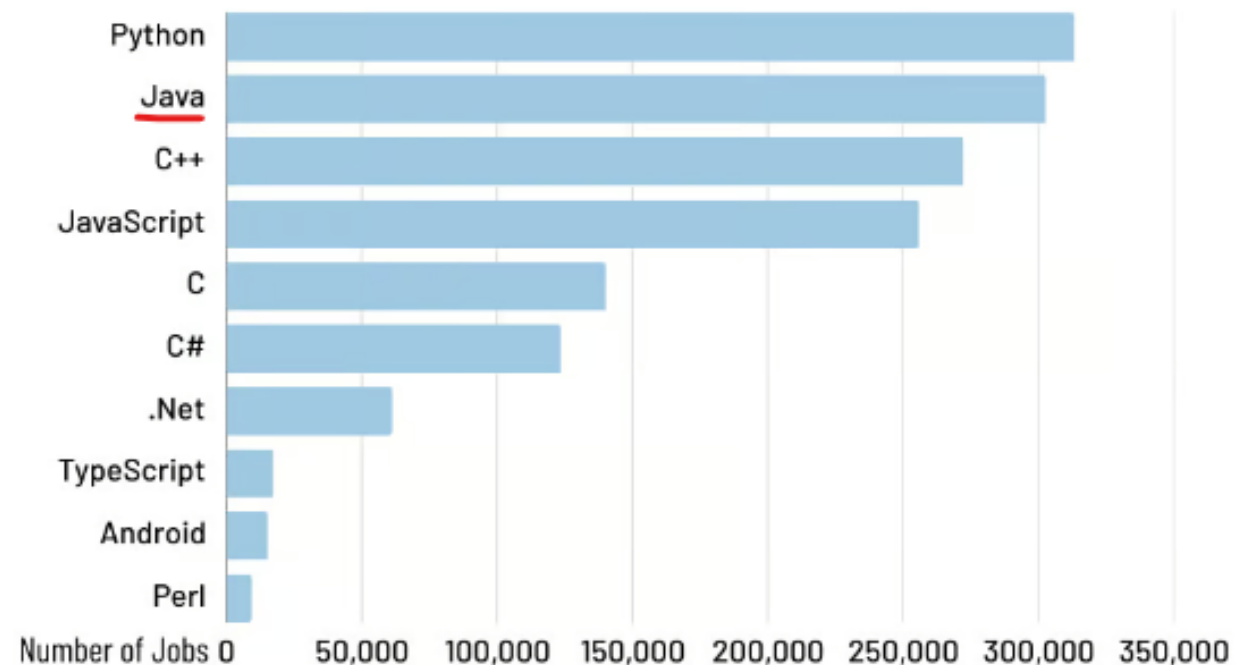
Tag 1	Variablen, Datentypen, Variablennamen, Verwendung, Arithmetik, Konvertierung, Kommentare, Technologie, IntelliJ Livedemo
Tag 2	Boolsche Ausdrücke, Kommentare, If-Abfragen, Switch-Case, Debugging
Tag 3	Arrays, (Do-)While-Schleife, For-Schleifen, Weiterführung Debugging
Tag 4	(statische) Methoden, Klassenvariablen, JavaDoc, Exceptions

Motivation

- Java ist eine der am weitesten verbreiteten und meistgenutzten Programmiersprachen weltweit!
- Als Java Entwickler hat man daher gute Job-Aussichten

Most in-demand programming languages of 2024

Based on LinkedIn job postings in the US



By: CodingNomads

Motivation

Anwendungsbereiche von Java:

- Java ist weltweit in vielen technischen Bereichen im Einsatz, vor allem in:
 - Web-Applikationen
 - Unternehmenssoftware
 - Android Apps



Motivation

- Bekannte Anwendungen, die mit Java implementiert sind:



NETFLIX



ebay



Hello World!



```
01 public class Application {  
02     public static void main(String[] args) {  
03         String greeting = „Hello World!“;  
04         System.out.println(greeting);  
05     }  
06 }
```

- Der Startpunkt für jedes Java Programm ist innerhalb der **Main-Methode**, die in einer **Klasse** stehen muss
- Schreibt euren Code innerhalb der geschweiften Klammern der Main-Methode, damit euer Programm funktioniert

Variablen

- Speicher für Werte, die sich ändern können
- Primitive Datentypen
 - Ganzzahlen
 - Fließkommazahlen
 - Wahrheitswerte
 - Zeichen
- Referenzdatentypen
 - Zeichenketten

Datentypen

- Zahlen im Computer sind endlich
- Ganzzahlen
 - **byte** (8 Bit / 1 Byte)
 - **short** (16 Bit / 2 Byte)
 - **int** (32 Bit / 4 Byte)
 - **long** (64 Bit / 8 Byte)
- Kommazahlen (Fließkommazahlen)
 - **float** (32 Bit / 4 Byte)
 - **double** (64 Bit / 8 Byte)
- Unterscheiden sich jeweils nur in ihrem Wertebereich

Datentypen

- Wahrheitswerte
 - **boolean**
 - **true** oder **false**
- 1 Zeichen (keine Zeichenkette)
 - **char**
 - 2 Byte lang
 - Darstellung als 16-Bit-Unicode-Wert
- Zeichenketten
 - **String**
 - Referenzdatentyp

Variablen – Wertebereich

Type	Länge		Wertebereich
	Byte	Bit	
boolean	–	–	true oder false
char	2	16	Unicode Zeichen
byte	1	8	-128 ... 127
short	2	16	-32.768 ... 32.767
int	4	32	-2.147.483.648 ... 2.147.483.647
long	8	64	$-2^{63} \dots 2^{63} - 1$
float	4	32	$\pm 1,4^{E-45} \dots \pm 3,4^{E+38}$
double	8	64	$\pm 4,9^{E-324} \dots \pm 1,7^{E+308}$

Variablennamen (Bezeichner)



- Vorgaben

- So **MÜSSEN** Namen sein, sonst gibt es Compiler-Fehler
- Erlaubte Zeichen: Buchstaben, Zahlen und _
- Erstes Zeichen darf keine Zahl sein

- Gesperrte Namen

- z.B.:

```
01 true, false, new      // richtig
02 double 1_variable;    // falsch
03 double variable1;     // richtig
```

- Konventionen

- So **SOLLTEN** Namen sein, jedoch kompiliert der Quelltext sobald die Formalen Voraussetzungen erfüllt sind
- Sinnvolle, aussagekräftige Namen wählen
- Keine Abkürzungen (außer sie sind geläufig)
- Substantive
- z.B.:

```
01 double payMoney;    // nicht aussagekräftig, kein Substantiv
02 double payTicket;   // kein Substantiv
03 double ticketPrice; // besser
```

- Konventionen

- Nur lateinische Zeichen (normales Alphabet), Zahlen und _
 - Kein ä, ö, ü, ß, ...
- Verwendung **EINER** Sprache, kein Gemisch oder „Denglisch“
 - **In der Praxis ist eine rein englische Schreibweise üblich!**

- lowerCamelCase-Schreibweise

camelCase bedeutet, dass der Bezeichner ohne Trennzeichen wie Leerzeile und Unterstrich angegeben werden und die folgenden Worte groß geschrieben werden. Die lower Variante beginnt das erste Wort klein geschrieben

- | | |
|----|-------------------------------|
| 01 | <code>int animalCount;</code> |
| 02 | <code>int bodyHeight;</code> |

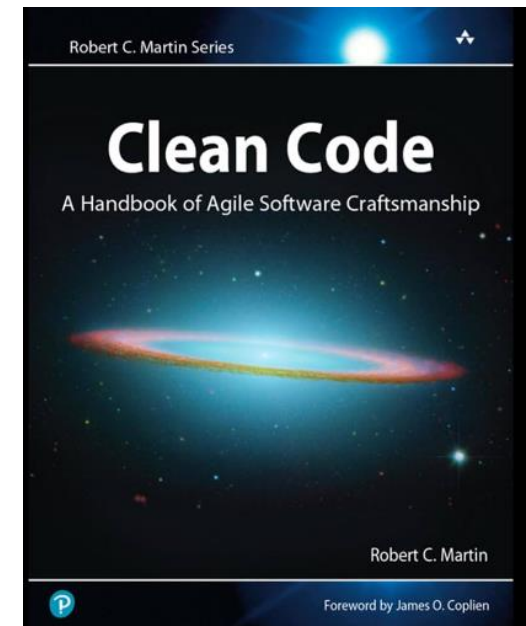
Variablennamen



“You should name a variable using the same care with which you name a first-born child.”

– **Robert C. Martin**, Clean Code: A Handbook of Agile Software Craftsmanship

- Robert C. Martin ist bekannt für seine Beiträge zu Prinzipien der **sauberen Softwarearchitektur**
- Variablen ordentlich zu benennen ist guter **Programmier-Stil!**



Deklaration

- Allgemeine Deklaration einer Variable:

```
01  datentyp name;
```

- Beispiele:

```
01  int age;  
02  float weight;  
03  double height;  
04  boolean isStudent;  
05  char gender;  
06  String name;
```


Wertzuweisung

- Der Variablen einen Wert zuweisen
 - Beim ersten Mal spricht man von **initialisieren**

```
01  name = value;    // Achtung! Nicht deklariert!
```

- Die Variable muss **deklariert** worden sein
- z.B.:

```
01  int age; age = 20;  
02  float weight; weight = 4.2f;  
03  double height; height = 1.74;  
04  boolean isStudent; isStudent = true;  
05  char gender; gender = 'm';  
06  String name; name = 'Douglas Adams';
```

Initialisierung

- Wert direkt beim Deklarieren auch initialisieren

```
01  datatype name = value;
```

- z.B.:

```
01  int age = 20;  
02  float weight = 4.2f;  
03  double height = 1.74;  
04  boolean isStudent = true;  
05  char gender = 'm';  
06  String name = 'Douglas Adams';
```

Ausgabe

- Sonst wäre es ziemlich langweilig

```
01 System.out.println(output);  
02 System.out.print(output);
```

- z.B.:

```
01 int value = 3;  
02 System.out.println(value);  
03  
04 System.out.println("Hello World");  
05  
06 String name = "World";  
07 System.out.print("Hello ");  
08 System.out.print(name);  
09 System.out.println();
```

Ausgabe

- Verknüpfung von Zeichenketten



- Verknüpfung durch den + -Operator

```
01 String name = „Hallo „ + „Welt“;
```

- Auch gemischt mit Zahlen möglich

```
01 int count = 5;  
02 String text = „count hat den Wert “ + count;
```

- Ausgabe:

```
01 System.out.println(„count ist: “ + count);  
02 System.out.println(„Hallo “ + „Student“);
```

Arithmetik

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
Inkrement	++	$a++$
Dekrement	--	$a--$
Modulo	%	$a \% b$

Das Ergebnis kann einer Variablen zugewiesen werden:

```
01  int result = 5 + 3;  
02  double division = 3.5 / (result - 1);
```

Arithmetik Beispiele

Beispiel 1:

```
01  int itemCount = 5;  
02  itemCount = itemCount + 1;
```

Ist äquivalent zu:

```
03  itemCount++;
```

Beispiel 2:

```
01  int totalPagesRead = 5;  
02  totalPagesRead = totalPagesRead + 5;
```

Ist äquivalent zu:

```
03  totalPagesRead += 5;
```

Modulo (Restwertbestimmung)

- Das Ergebnis des Modulo ist der Rest der Division:

$$26/5 = 5 \text{ Rest } 1 \Rightarrow 26 \% 5 = 1$$

$$30/2 = 15 \text{ Rest } 0 \Rightarrow 30 \% 2 = 0$$

$$65/10 = 6 \text{ Rest } 5 \Rightarrow 65 \% 10 = 5$$

$$29/30 = 0 \text{ Rest } 29 \Rightarrow 29 \% 30 = 29$$

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
Inkrement	++	$a++$
Dekrement	--	$a--$
Modulo	%	$a \% b$

In- bzw. Dekrement

- Erhöht bzw. verringert den Wert einer Variablen um 1

```
01  int scorePlayerOne = 42;  
02  int scorePlayerTwo = 38;  
03  scorePlayerOne++;  
    // identisch: scorePlayerOne=scorePlayerOne+1  
04  scorePlayerTwo--;  
    // identisch: scorePlayerTwo-=scorePlayerTwo-1
```

Welchen Wert beinhalten scorePlayerOne und scorePlayerTwo?

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
Inkrement	++	$a++$
Dekrement	--	$a--$
Modulo	%	$a \% b$

In- bzw. Dekrement

- Erhöht bzw. verringert den Wert einer Variablen um 1

```
01  int scorePlayerOne = 42;  
02  int scorePlayerTwo = 38;  
03  scorePlayerOne++;  
    // identisch: scorePlayerOne=scorePlayerOne+1  
04  scorePlayerTwo--;  
    // identisch: scorePlayerTwo=scorePlayerTwo-1
```

Welchen Wert beinhalten scorePlayerOne und scorePlayerTwo?

```
05  System.out.println(„scorePlayerOne: “ + scorePlayerOne);  
06  System.out.println(„scorePlayerTwo: “ + scorePlayerTwo);
```



scorePlayerOne: 43
scorePlayerTwo: 37

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
Inkrement	++	$a++$
Dekrement	--	$a--$
Modulo	%	$a \% b$

Präinkrement & Postinkrement



- Präinkrement (Prä → Davor)

```
01  int preScore = 23;  
02  System.out.println(++preScore)
```

Wie lautet die Ausgabe und welchen Wert hat preScore tatsächlich?

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
(Prä-) Inkrement	++	$++a$
(Prä-) Dekrement	--	$--a$
Modulo	%	$a \% b$

Präinkrement & Postinkrement



- Präinkrement (Prä → Davor)

```
01  int preScore = 23;  
02  System.out.println(++preScore)
```

Wie lautet die Ausgabe und welchen Wert hat preScore tatsächlich?

Ausgabe: **24**

Wert: preScore = 24

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
(Prä-) Inkrement	++	$a++$
(Prä-) Dekrement	--	$a--$
Modulo	%	$a \% b$

Der Wert der Variablen wird hier zuerst erhöht, **bevor** sie ausgegeben wird

Präinkrement & Postinkrement



- Postinkrement (Post → Danach)

```
01  int postScore = 23;  
02  System.out.println(postScore++)
```

Wie lautet die Ausgabe und welchen Wert hat postScore tatsächlich?

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
(Post-) Inkrement	++	$a++$
(Post-) Dekrement	--	$a--$
Modulo	%	$a \% b$

Präinkrement & Postinkrement

- Postinkrement (Post → Danach)

```
01  int postScore = 23;  
02  System.out.println(postScore++)
```

Wie lautet die Ausgabe und welchen Wert hat postScore tatsächlich?

Ausgabe: **23**

Wert: postScore = 24

Bezeichnung	Operator	Anwendung
Addition	+	$a + b$
Subtraktion	-	$a - b$
Multiplikation	*	$a * b$
Division	/	a / b
(Post-) Inkrement	++	$a++$
(Post-) Dekrement	--	$a--$
Modulo	%	$a \% b$

Die Variable wird zuerst ausgegeben (23) und **danach** wird ihr Wert erhöht (24)

Integer-Division

- Integer ist der Datentyp für Ganzzahlen
- Bei der Division von Ganzzahlen entsteht meist eine Kommazahl (z.B $7:2 = 3.5$)
- Da der Datentyp `int` aber keine Kommazahlen darstellen kann, wird in Java bei einer Integer-Division **auf den nächstkleineren ganzen Wert abgerundet**

Beispiel:

```
01  int cakeSlices = 7;  
02  int guests = 2;  
03  double cakeSlicesPerGuest = cakeSlices / guests;
```

Welchen Wert beinhaltet `cakeSlicesPerGuest`?

Integer-Division

- Integer ist der Datentyp für Ganzzahlen
- Bei der Division von Ganzzahlen entsteht meist eine Kommazahl (z.B $7:2 = 3.5$)
- Da der Datentyp `int` aber keine Kommazahlen darstellen kann, wird in Java bei einer Integer-Division **auf den nächstkleineren ganzen Wert abgerundet**

Beispiel:

```
01  int cakeSlices = 7;  
02  int guests = 2;  
03  double cakeSlicesPerGuest = cakeSlices / guests;
```

Welchen Wert beinhaltet `cakeSlicesPerGuest`?

```
04  System.out.println("cakeSlicesPerGuest: "  
    + cakeSlicesPerGuest);
```



cakeSlicesPerGuest: 3.0

In diesem Kontext hätten wir das Ergebnis 3.5 erwartet, aber durch die Integer Division erhalten wir 3.0 und erzeugen damit ungenaue Informationen!

*Um das zu vermeiden, müssen wir korrekte **Typ-Konvertierungen** anwenden.*

Implizite Konvertierung

- Einige Typen können ohne Probleme in andere umgewandelt werden

byte → **short** → **int** → **long** → **double**
byte → **short** → **int** → **float** → **double**

```
01  int x = 42;  
02  float y = x;  
03  
04  byte a = 123;  
05  short b = a;  
06  long c = b;  
07  double d = c;    // d = 123.0
```

- „Kleinere“ Datentypen können ohne Probleme in „größere“ Datentypen konvertiert werden, da bei dieser Konvertierung keine Informationen verloren gehen
- Java konvertiert die Datentypen automatisch, ohne dass wir extra Code hinzugeben müssen (daher wird hier **implizit** konvertiert)

Explizite Konvertierung

- Variablenwerte können umgewandelt werden
→ Explizites „Casten“

Beispiel 1:

```
01  int price = 42;  
02  double priceExact = (double)price;
```

Welchen Wert beinhaltet priceExact?

```
04  System.out.println(„priceExact: “  
    + priceExact);
```

➡ priceExact: 42.0

Wir können Java hier **explizit** anweisen, die Variable `price` in den Datentyp `double` zu konvertieren

Explizite Konvertierung

- Noch ein Beispiel:

```
01 double averageScore = 512.6;  
02 int roundedScore = (int)averageScore;
```

Welchen Wert beinhaltet roundedScore?

Explizite Konvertierung

- Noch ein Beispiel:

```
01 double averageScore = 512.6;  
02 int roundedScore = (int)averageScore;
```

Welchen Wert beinhaltet roundedScore?

```
04 System.out.println("roundedScore: "  
    + roundedScore);
```



roundedScore: 512

- Die Variable `averageScore` wird hier explizit in den Datentyp `int` konvertiert
- Dabei wird alles, was noch dem Komma steht „abgeschnitten“
- Diese Daten gehen dann verloren

Zurück zum Divisionsproblem

```
01  int cakeSlices = 7;  
02  int guests = 2;  
03  double cakeSlicesPerGuest = cakeSlices / guests;
```

- Bei Rechnungen wird in den bestmöglichen Typ gecastet
- So funktioniert es:

```
04  double cakeSlicesPerGuest = (double)cakeSlices / guests;
```

Welchen Wert beinhaltet cakeSlicesPerGuest in diesem Fall?

Zurück zum Divisionsproblem

```
01  int cakeSlices = 7;  
02  int guests = 2;  
03  double cakeSlicesPerGuest = cakeSlices / guests;
```

- Bei Rechnungen wird in den bestmöglichen Typ gecastet
- So funktioniert es:

```
04  double cakeSlicesPerGuest = (double)cakeSlices / guests;
```

Welchen Wert beinhaltet cakeSlicesPerGuest in diesem Fall?

```
04  System.out.println(„cakeSlicesPerGuest: “  
    + cakeSlicesPerGuest);
```



cakeSlicesPerGuest: 3.5

- Die Variable `cakeSlices` wird hier explizit von `int` zu `double` konvertiert
- Die Division wird als Gleitkommaoperation durchgeführt, da einer der Operanden ein `double` ist

Zurück zum Divisionsproblem

- Allgemein gilt für die Integer-Division:

Möchte man eine genaue Division mit Nachkommastellen erreichen, dann muss man mindestens einen der Operanden **explizit** in einen **double** oder **float** umwandeln, um eine Fließkomma-Division zu erhalten, so wie hier:

```
04 double cakeSlicesPerGuest = (double)cakeSlices / guests;
```

Kommentare

- Werden verwendet, um Code von der Verwendung auszunehmen oder Kommentare zu hinterlassen. Wenn wir euch auffordern etwas auszukommentieren reden wir hiervon.

Einzeilige Kommentare:

```
01  int number; // Kommentar
02  // int number = 3;
03  number = 12;
```

Mehr zu Kommentaren an Tag 2!

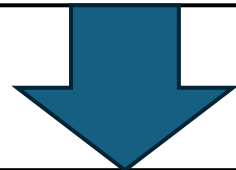
Technologie



Quellcode (Java)

= Der vom Programmierer
geschriebene Code
= Vom Menschen lesbar, aber nicht
direkt vom Computer ausführbar

Quellcode (Java)



Compiler

Zwischencode (Bytecode)

= Der vom Programmierer
geschriebene Code
= Vom Menschen lesbar, aber nicht
direkt vom Computer ausführbar

= Zwischencode, der von einer
Laufzeitumgebung (JVM) interpretiert
und weiter übersetzt werden kann

Quellcode (Java)

= Der vom Programmierer geschriebene Code
= Vom Menschen lesbar, aber nicht direkt vom Computer ausführbar

Compiler

Zwischencode (Bytecode)

= Zwischencode, der von einer Laufzeitumgebung (JVM) interpretiert und weiter übersetzt werden kann

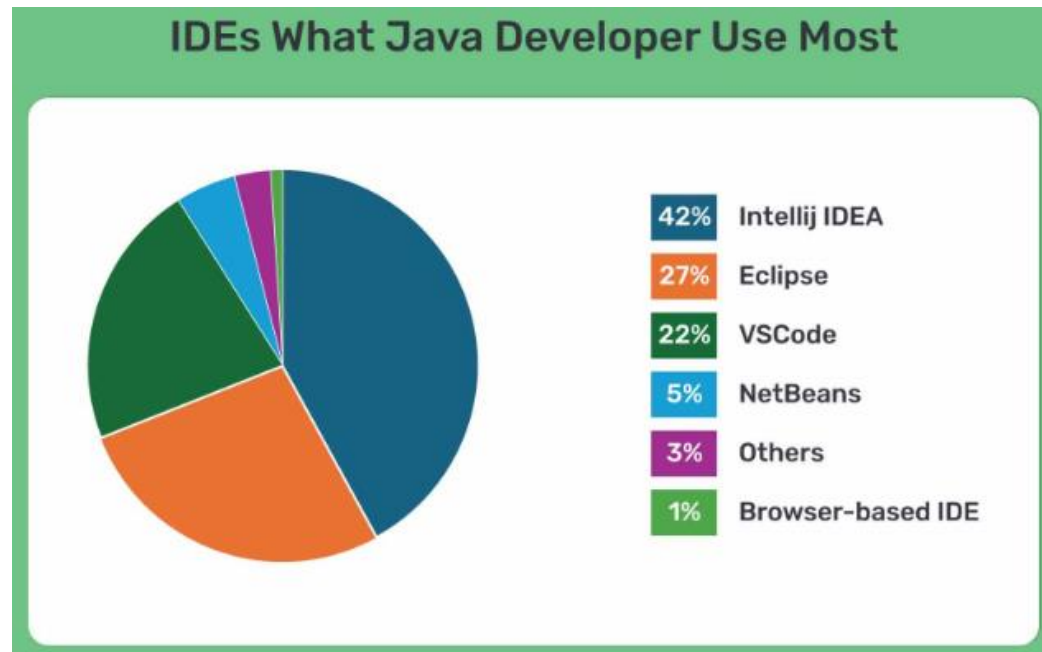
Laufzeitumgebung (JVM)

Zwischencode (Bytecode)

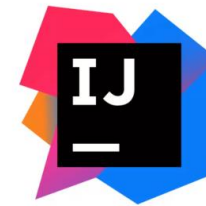
= Der Binäre Code, der direkt von der CPU ausgeführt werden kann
= Vom Menschen nicht lesbar, aber vom Computer ausführbar

Entwicklungsumgebungen

- Das wichtigste Werkzeug, um eine Applikation in Java zu entwickeln ist eine gute Entwicklungsumgebung / IDE (= Integrated Development Environment)

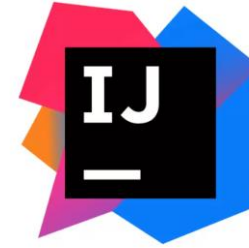


- Die am weitesten verbreitete IDE ist momentan IntelliJ, gefolgt von Eclipse und VSCode



- Wir werden euch in diesem Kurs IntelliJ vorstellen

IntelliJ Einführung



➤ Live Demo

